Practical No. 5 : Use BFS to solve a planning problem 8 puzzle

```
from collections import deque
GOAL = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

def get_neighbors(state):
    state = [list(row) for row in state]

    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    neighbors = []

    for dx, dy in moves:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]

            new_state[x][y], new_state[nx][ny] = (
                new_state[nx][ny],
                new_state[x][y]
            )
```

```
            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def print_path(parent, goal):
    path = []
    current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()

    print("\nSolution Path:\n")

    for step, state in enumerate(path):
        print("Step", step)

        for row in state:
            print(row)

        print()

    print("Total Moves =", len(path) - 1)

def bfs(initial):
    queue = deque([initial])
```

```
    visited = set([initial])
    parent = {initial: None}

    while queue:
        current = queue.popleft()

        if current == GOAL:
            print_path(parent, current)
            return

        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)

    print("No Solution Exists")

initial = ((1, 2, 3),
           (0, 4, 6),
           (7, 5, 8))

bfs(initial)
```